SANJIVANI UNIVERSITY

Kopargaon, Maharashtra

B.Tech – Artificial Intelligence & Data Science

DevOps Practical Assignment

Technical Project Report & Practical Record

Enterprise Smart Library Management Platform

Git • Flask • Pytest • Jenkins CI • Docker • Kubernetes (Minikube)

Student Name	Om Bhosale
PRN	2125UDSM1037
Program	B.Tech – Artificial Intelligence & Data Science
University	Sanjivani University, Kopargaon
Assignment ID	5253
Date of Submission	12 October 2026

Contents

1. Introduction and Objectives
2. Environment and Tools
3. Topic 1 – Project Setup and Repository Structure
4. Topic 2 – Flask Application
5. Topic 3 – Automated Testing with Pytest and Coverage
6. Topic 4 – Jenkins CI Pipeline
7. Topic 5 – Docker Containerization
8. Topic 6 – Kubernetes Orchestration on Minikube
9. Final Step – Shutdown and Preservation
10. Challenges Encountered and Resolutions
11. Conclusion
List of Figures

1. Introduction and Objectives

This report documents the hands-on implementation of an automated DevOps delivery pipeline for the Enterprise Smart Library Management Platform, a REST API built with Python and Flask. The work follows the step-by-step engineering guide supplied with the assignment and covers the complete lifecycle of the application: development, automated testing, continuous integration, containerization and orchestration.

Every result reported here is supported by a screenshot captured during the practical on 12 October 2026. The objectives were to:

- Prepare a reproducible Linux development environment (Ubuntu on WSL2) with Python, pip, virtualenv and Git.
- Build and verify a modular Flask REST API with a health-check endpoint.
- Execute an automated Pytest suite and measure code coverage.
- Automate build and test execution using a Jenkins pipeline.
- Package the application as a versioned, non-root Docker image and run it as a healthy container.
- Deploy three replicas to Kubernetes (Minikube), expose them through a NodePort service, and demonstrate self-healing.

2. Environment and Tools

All commands were executed in an Ubuntu terminal running through WSL on Windows. Jenkins was run as a Windows installation and accessed through its web interface on port 8080. The versions below were taken directly from the terminal and browser output.

Component	Version / Detail	Evidence
Operating system	Ubuntu (WSL2) on Windows	Figures 1–2
Python (host)	3.14.4	Figures 1–2
Python (container)	3.11-slim base image	Figure 12
Git	2.53.0	Figures 1–2
pip	26.2.1 (virtualenv) / 25.1.1 (system package)	Figures 1–2
Flask / Werkzeug	3.0.3 / 3.0.3	Figure 8
Pytest / pytest-cov	9.1.1 / 5.0.0	Figure 8
Jenkins	2.580.1	Figures 10–11
Docker	Image built and container run with Docker Engine	Figure 12
Minikube	v1.39.0 (Docker driver)	Figure 12

3. Topic 1 – Project Setup and Repository Structure

3.1 Installing prerequisites

The system package index was refreshed and the required tooling (Python 3, pip, venv, Git, curl, wget and jq) was installed and verified with the following commands:

```
# Update local APT package indexes
sudo apt-get update && sudo apt-get upgrade -y

# Install Python, pip, virtualenv, and Git
sudo apt-get install -y python3 python3-pip python3-venv git curl wget jq

# Verify tool installations
git --version
python3 --version
pip3 --version
```

The run reported that every package was already at its newest version, with 0 upgraded and 0 newly installed. The verification commands returned Git 2.53.0 and Python 3.14.4. Figure 1 shows the full command sequence and Figure 2 shows the repeat verification, including the pip version.

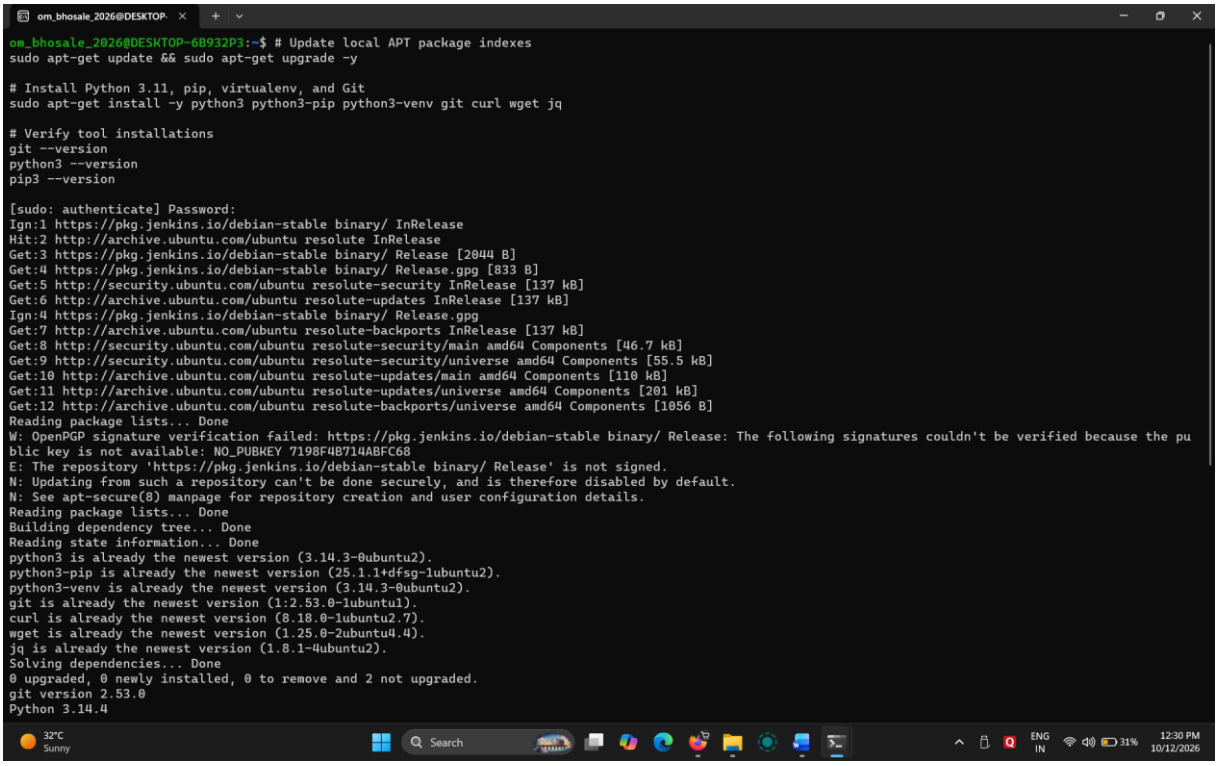


Figure 1: *Prerequisite installation and tool verification (apt-get update, install, git and python3 versions)*

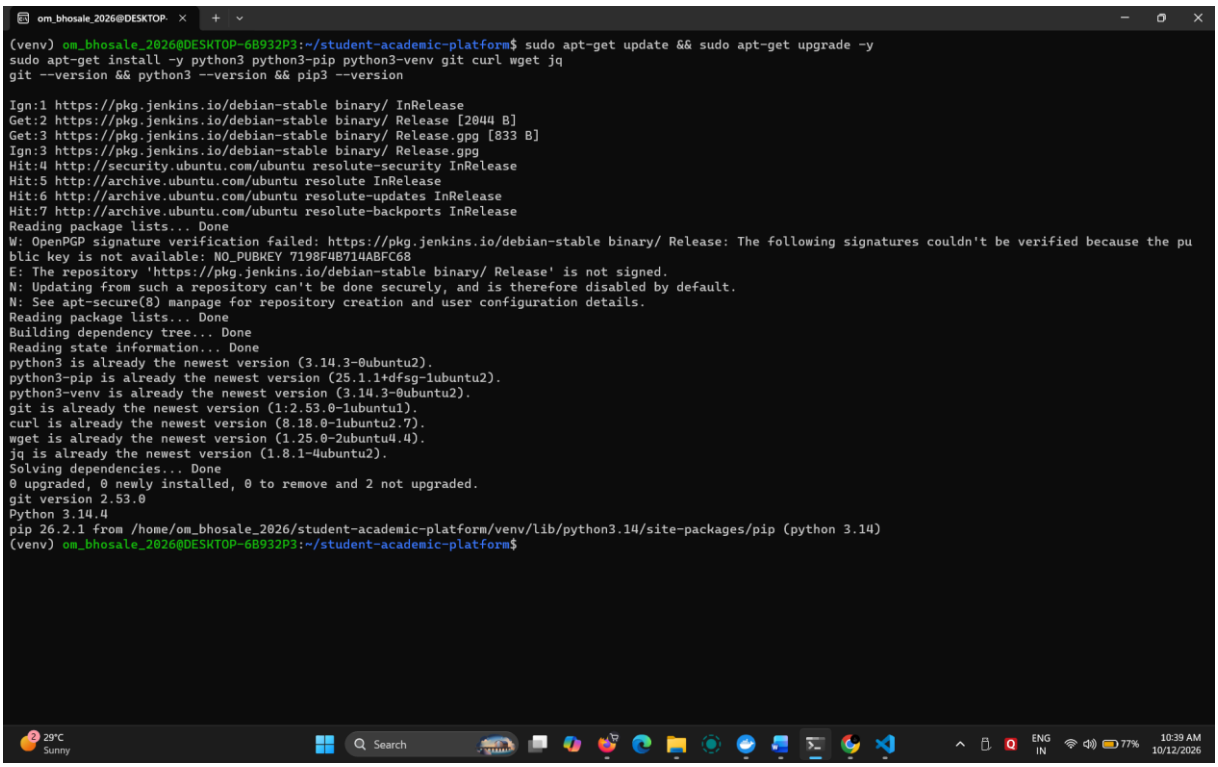


Figure 2: *Repeat verification inside the project virtual environment, showing git 2.53.0, Python 3.14.4 and pip 26.2.1*

Observation: the update step printed a warning that the Jenkins apt repository could not be verified because its public key was not available (NO_PUBKEY). The warning only affects that third-party repository; the Ubuntu repositories updated normally and the required packages were present. Section 10 records this item.

3.2 Project directory structure

The project is organised as a multi-tier structure that separates the web layer, business logic, data layer, tests and deployment configuration. The project was opened with cd ~/library-management-platform and listed with ls.

```
library-management-platform/
├── app/
│   ├── __init__.py          # Flask application factory
│   ├── routes/api.py        # REST API endpoints
│   ├── services/            # Business logic (library_service.py)
│   └── models/              # Book and User entities
├── database/db.py           # Thread-safe in-memory data store
├── tests/                   # Unit and integration tests
├── k8s/
│   ├── deployment.yaml      # Deployment: 3 replicas, probes
│   └── service.yaml         # Service: NodePort 30080
├── Jenkinsfile              # Declarative CI pipeline
├── Dockerfile               # Container build recipe
├── .dockerignore
├── requirements.txt         # Python dependencies
├── run.py                   # Application launcher
└── README.md
```

3.3 Version control – GitHub repository

The source code is version-controlled with Git and hosted in a public GitHub repository, ombhosale0906/student-academic-platform-. The repository root contains the same layout described

above: the app, database, k8s and tests folders together with .dockerignore, .gitignore, Dockerfile, Jenkinsfile, README.md, requirements.txt and run.py. Figure 3 shows the repository page.

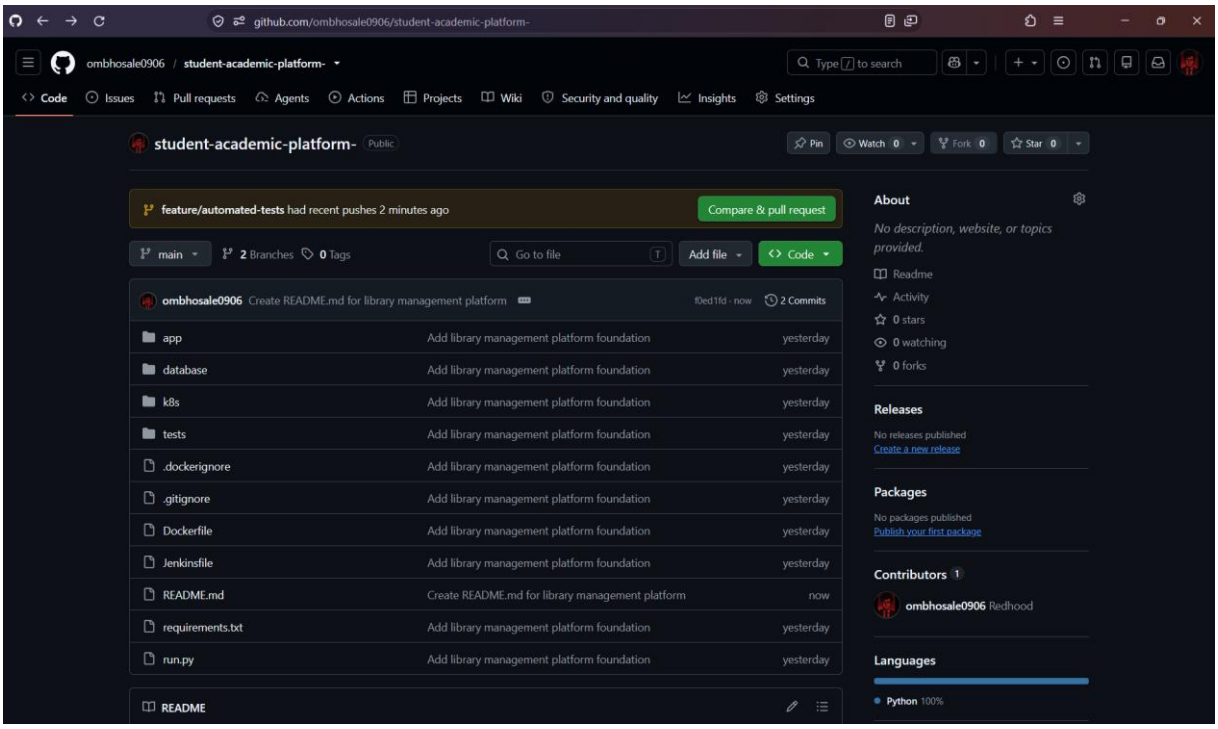


Figure 3: *GitHub repository page showing the project folders and files, 2 branches (main and feature/automated-tests) and 2 commits*

The repository holds 2 commits and 2 branches: main, and feature/automated-tests, which had just received pushes (GitHub offers “Compare & pull request” for it). The folders were committed with the message “Add library management platform foundation”, and the latest commit, “Create README.md for library management platform”, added the README. Python makes up 100% of the repository languages. Note that the repository name uses the student-academic-platform prefix while its contents are the Library Management Platform project.

4. Topic 2 – Flask Application

The application is built with Python and Flask using the application-factory pattern. create_app() in app/__init__.py registers a Blueprint with the URL prefix /api, and run.py starts the server on host 0.0.0.0, port 5000, so that it can be reached from inside a container. Business rules live in the service layer, while database/db.py provides an in-memory store seeded with sample users (U101, U102) and books (B001, B002).

4.1 API endpoints

Method	Endpoint	Purpose
GET	/api/health	Liveness/readiness check used by Docker and Kubernetes
POST	/api/users	Register a user (validation errors return HTTP 400)
GET	/api/users/<user_id>	Fetch a user (HTTP 404 if not found)
GET	/api/books	List or search books using the optional q parameter
POST	/api/books	Add a book to the catalogue

Method	Endpoint	Purpose
POST	/api/borrow, /api/return	Check a book out or return it

Figure 4 shows the route module in the terminal. The health route returns status UP, the service name and version 1.0.0 with HTTP 200; the user and book routes delegate to LibraryService and map ValueError to HTTP 400 and missing users to HTTP 404.

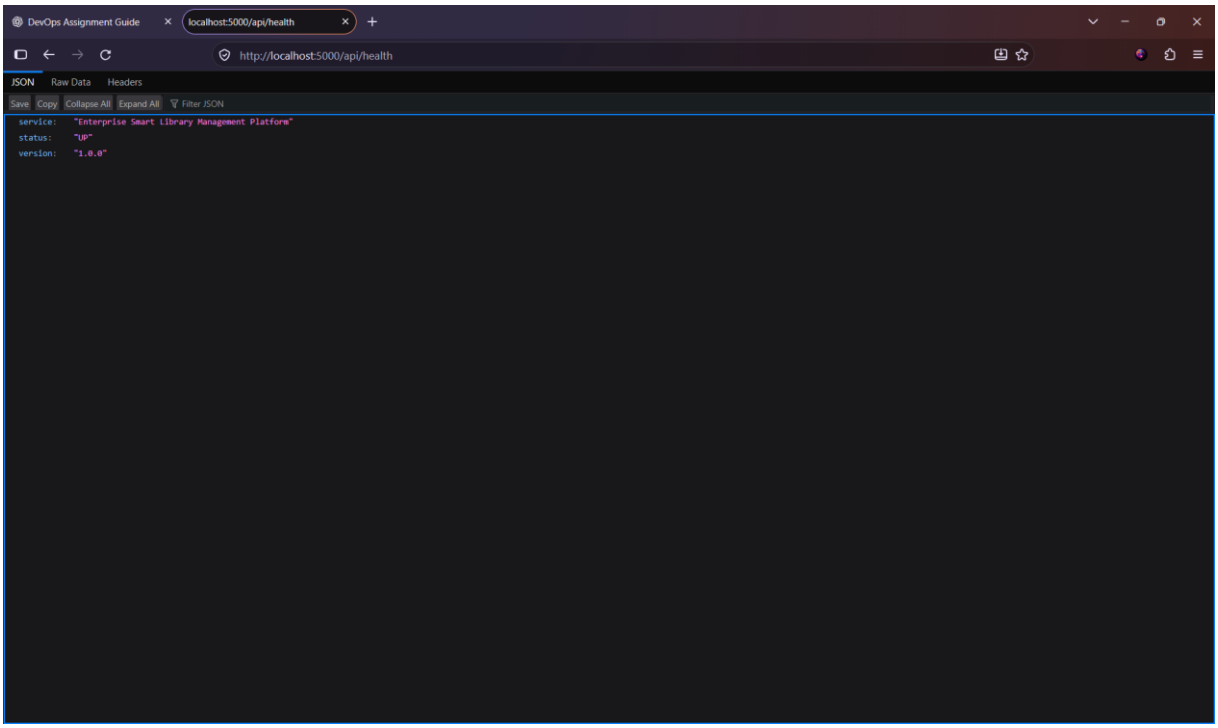


Figure 4: *Source of app/routes/api.py: health check, user registration and lookup, and book listing routes*

4.2 Verifying the running application

With the application running on port 5000, three endpoints were opened in the browser to confirm that the API responds correctly.

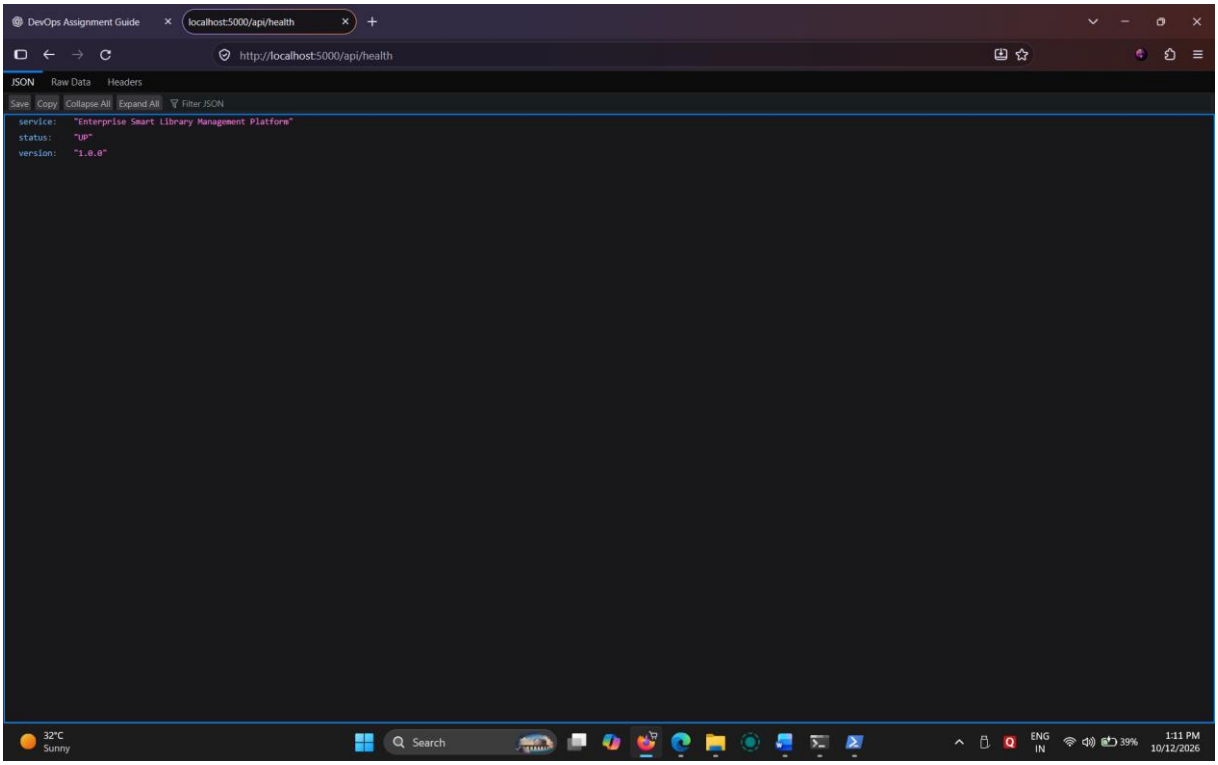


Figure 5: *GET /api/health returns service "Enterprise Smart Library Management Platform", status "UP", version "1.0.0"*

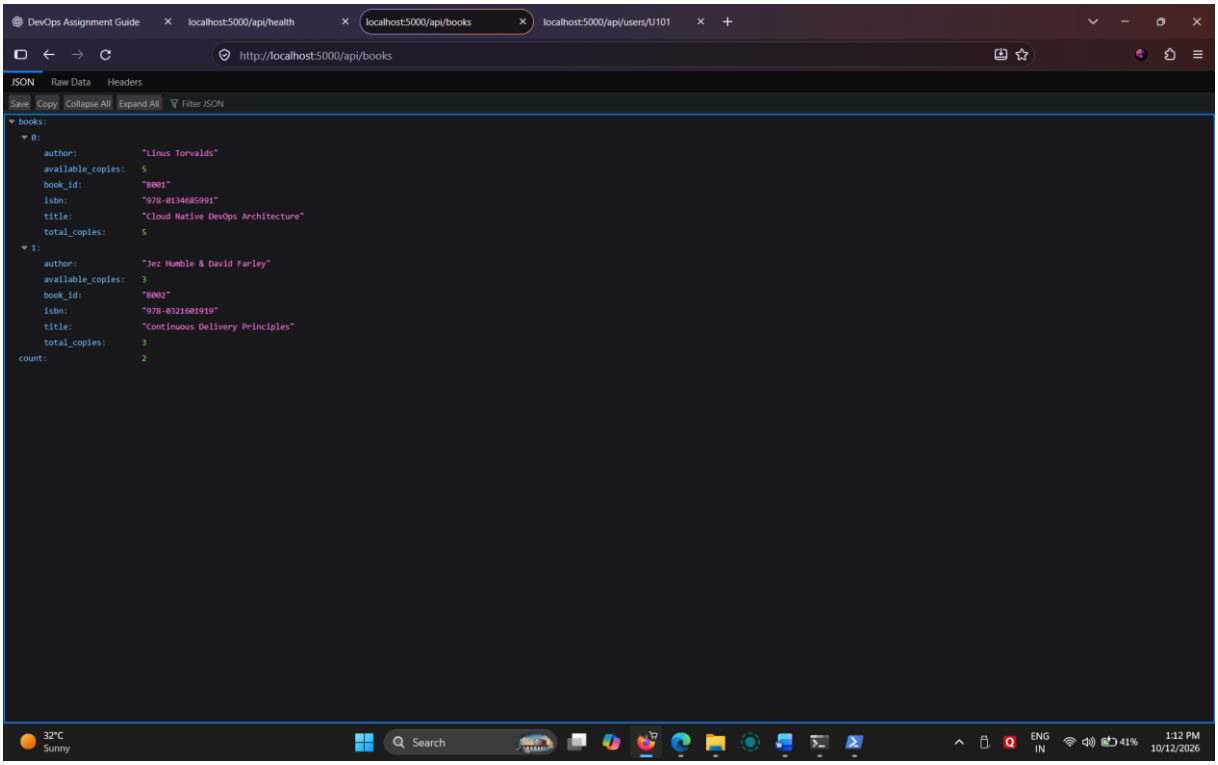


Figure 6: *GET /api/books returns count 2 with books B001 (Cloud Native DevOps Architecture, 5 copies) and B002 (Continuous Delivery Principles, 3 copies)*

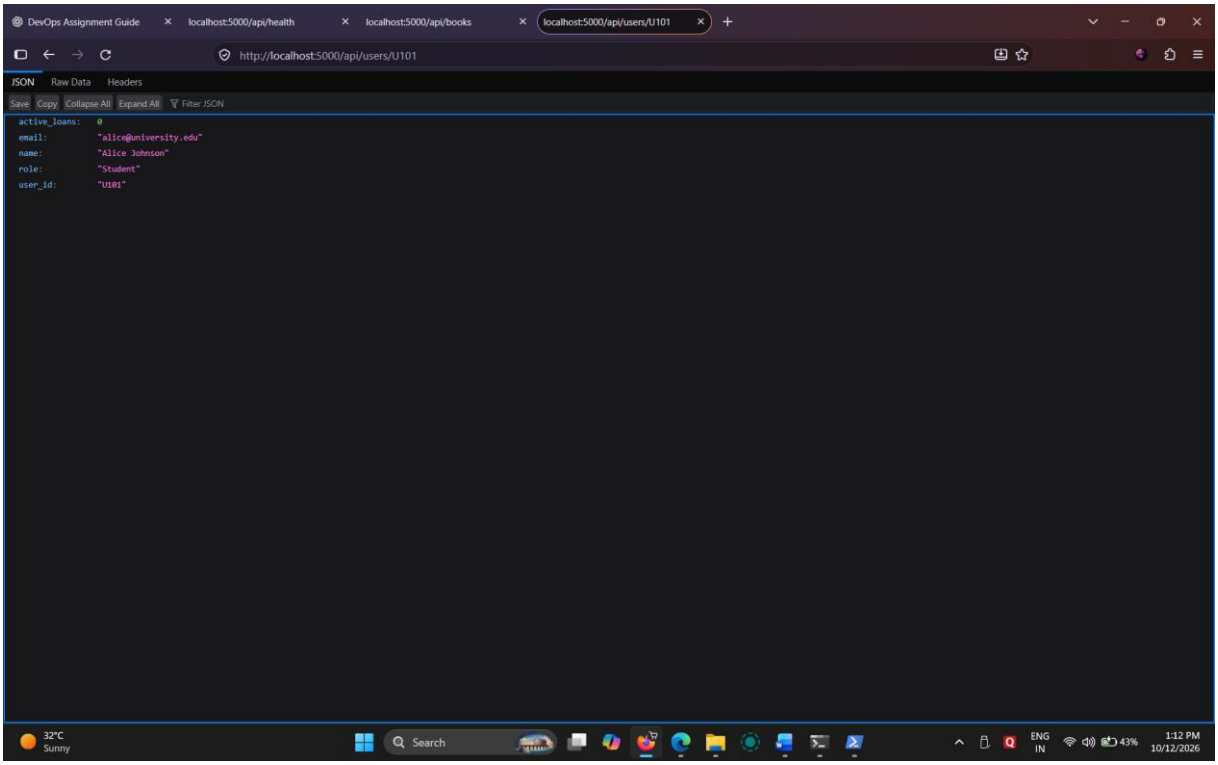


Figure 7: *GET /api/users/U101 returns Alice Johnson, role Student, with 0 active loans*

Result: the health endpoint confirms the application starts and answers requests, and the data endpoints return the seeded records exactly as defined in the data store.

5. Topic 3 – Automated Testing with Pytest and Coverage

A virtual environment was created, the dependencies in requirements.txt were installed, and the complete test suite was run with verbose output, a terminal coverage report and a JUnit XML report:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

pytest -v --cov=app --cov-report=term-missing --junitxml=reports/test-results.xml tests/
```

Option	Meaning
-v	Prints each test name with its result
--cov=app	Measures how much of the app/ package the tests execute
--cov-report=term-missing	Lists the line numbers that were not executed
--junitxml=reports/test-results.xml	Writes a JUnit report that Jenkins can publish

Note: the screenshots for this topic were captured in the student-academic-platform workspace, the parallel student track of the assignment, which uses the same Flask, Pytest and coverage toolchain. The dependencies were already satisfied, so pip reported “Requirement already satisfied” for every package.

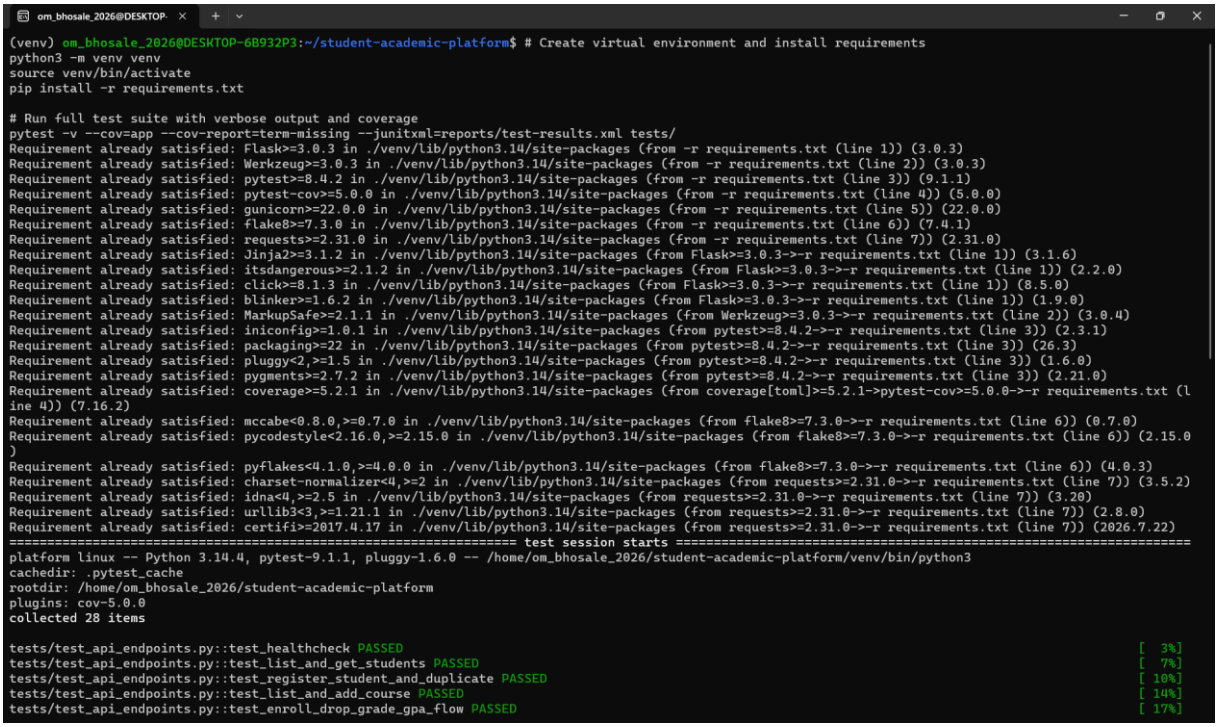


Figure 8: *Virtual-environment setup, dependency check and start of the Pytest session (28 items collected, Python 3.14.4, pytest 9.1.1)*

5.1 Results

All 28 collected tests passed in 0.28 seconds. They cover API endpoints (6 tests), course service (4), enrollment rules (7), grades and GPA (5) and student service (6). Business rules exercised include duplicate enrollment, credit-limit enforcement, schedule conflicts, course capacity, GPA weighting by credits and invalid input handling.

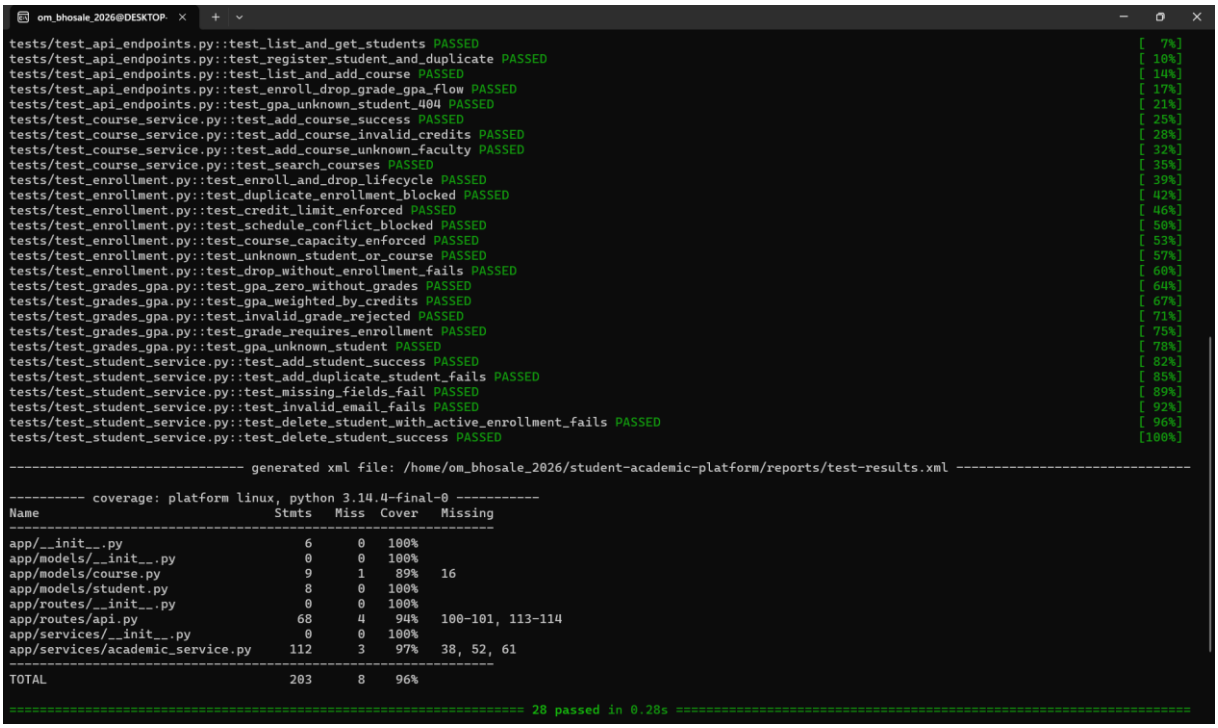


Figure 9: *Pytest results: 28 passed in 0.28 s, with the coverage report (96% total) and generated reports/test-results.xml*

Module	Stmts	Miss	Cover	Missing lines
app/__init__.py	6	0	100%	–
app/models/__init__.py	0	0	100%	–
app/models/course.py	9	1	89%	16
app/models/student.py	8	0	100%	–
app/routes/__init__.py	0	0	100%	–
app/routes/api.py	68	4	94%	100-101, 113-114
app/services/__init__.py	0	0	100%	–
app/services/academic_service.py	112	3	97%	38, 52, 61
TOTAL	203	8	96%	

Overall statement coverage is 96% (195 of 203 statements executed), and the JUnit XML file was generated for use by the CI server.

6. Topic 4 – Jenkins CI Pipeline

Jenkins automates the repeatable part of delivery: it prepares the environment, installs dependencies and runs the tests on every build. A pipeline job named Library-Management-Platform was created on a Jenkins 2.580.1 server running at http://localhost:8080.

6.1 Pipeline stages and result

Stage	Duration	Purpose
Check Python	0.57 s	Confirms the Python runtime is available
Install Dependencies	1 min 51 s	Installs the packages from requirements.txt
Run Automated Tests	1 s	Executes the Pytest suite
Post Actions	0.1 s	Prints “Library Management Platform pipeline completed.”

Build #2 finished successfully in 1 min 54 s, and every stage shows a green check mark in the stage view.

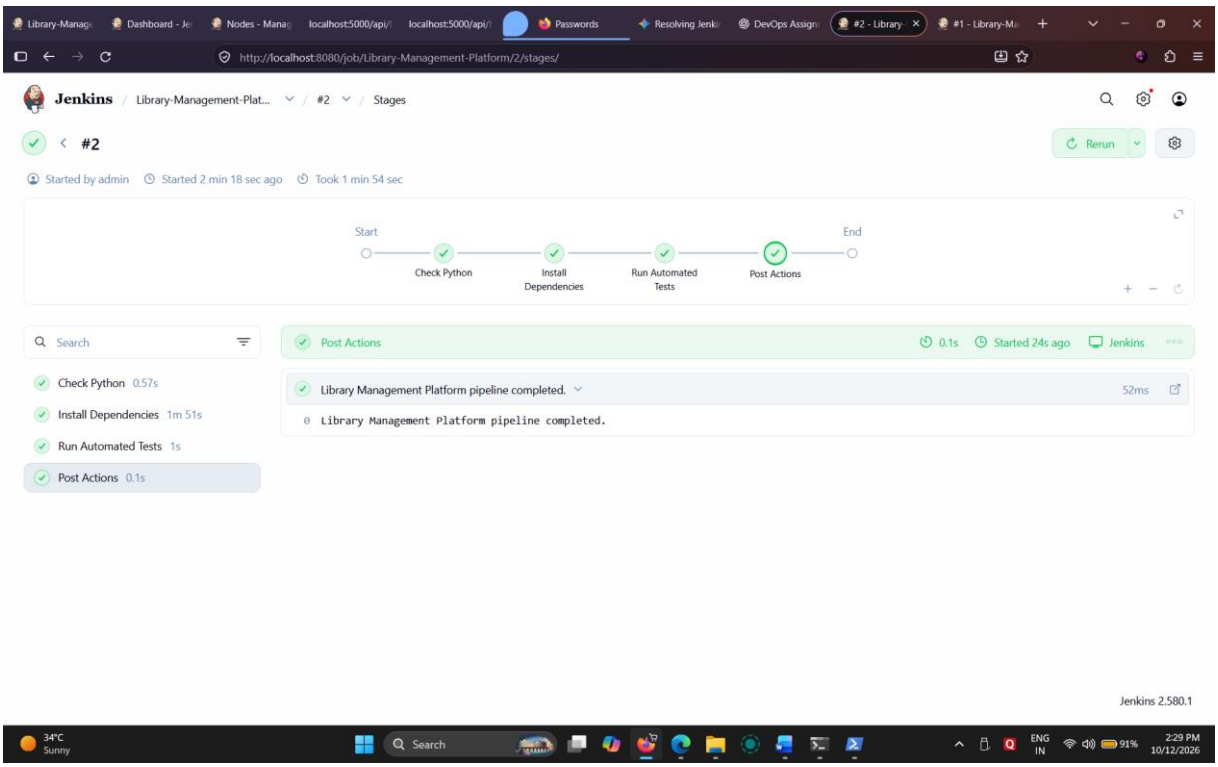


Figure 10: *Jenkins stage view for build #2: all four stages passed (Started by admin, took 1 min 54 sec)*

6.2 Build history

The job page records the build history. Build #1 (8:53 AM) failed; after troubleshooting, build #2 (8:56 AM) succeeded. The permalinks confirm build #2 as the last stable, successful and completed build, and build #1 as the last failed build.

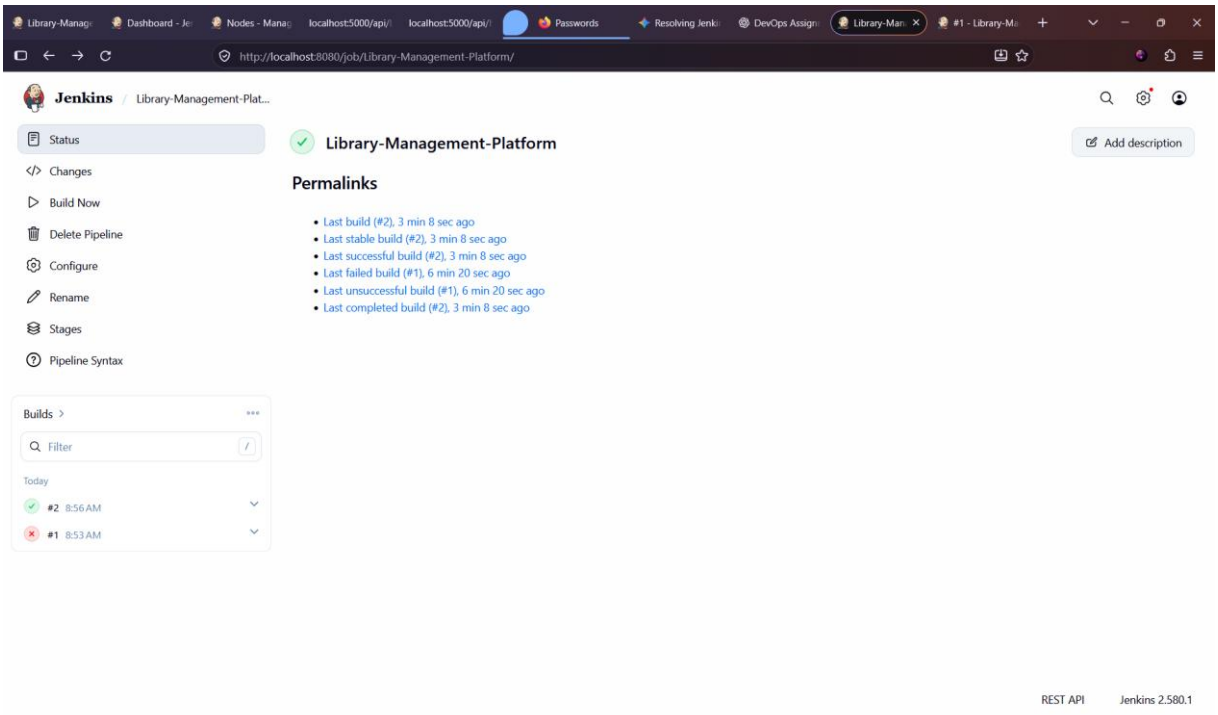


Figure 11: *Library-Management-Platform job page: build #2 successful, build #1 failed, with permalinks*

Note: the reference Jenkinsfile in the guide also defines linting, Docker build and Kubernetes deployment stages. In this practical, Jenkins performed the CI stages (environment check, dependency installation

and test execution); the Docker and Kubernetes stages were then carried out and verified manually in the Ubuntu terminal, as shown in Topics 5 and 6.

7. Topic 5 – Docker Containerization

Containerization packages the application, its runtime and its dependencies into one immutable image. The Dockerfile uses the python:3.11-slim base image, installs curl for the health check, installs the Python requirements before copying the source code (so the dependency layer is cached), and creates a non-root user appuser (UID 1000) to run the application.

Step	Command	Purpose
1. Build	docker build -t library-management-platform:v1.0.0 .	Create the versioned image
2. List images	docker images	Confirm the image exists locally
3. Run	docker run -d -p 5000:5000 --name library-container library-management-platform:v1.0.0	Start the container in the background and map port 5000
4. Verify	docker ps	Check container status
5. Test	curl http://localhost:5000/api/health	Call the health endpoint
6. Stop	docker stop library-container	Stop the container, keeping the image

Figure 12 shows the build and run sequence. The build finished in 38.9 s (12/12 steps) and the image library-management-platform:v1.0.0 (ID c017630141ce) appears in docker images with 246 MB disk usage (59.8 MB content size). The container library-container started from the image and, 25 seconds later, docker ps reported it as Up (healthy) with 0.0.0.0:5000 mapped to container port 5000. Calling curl on /api/health returned {"service":"Enterprise Smart Library Management Platform","status":"UP","version":"1.0.0"}.

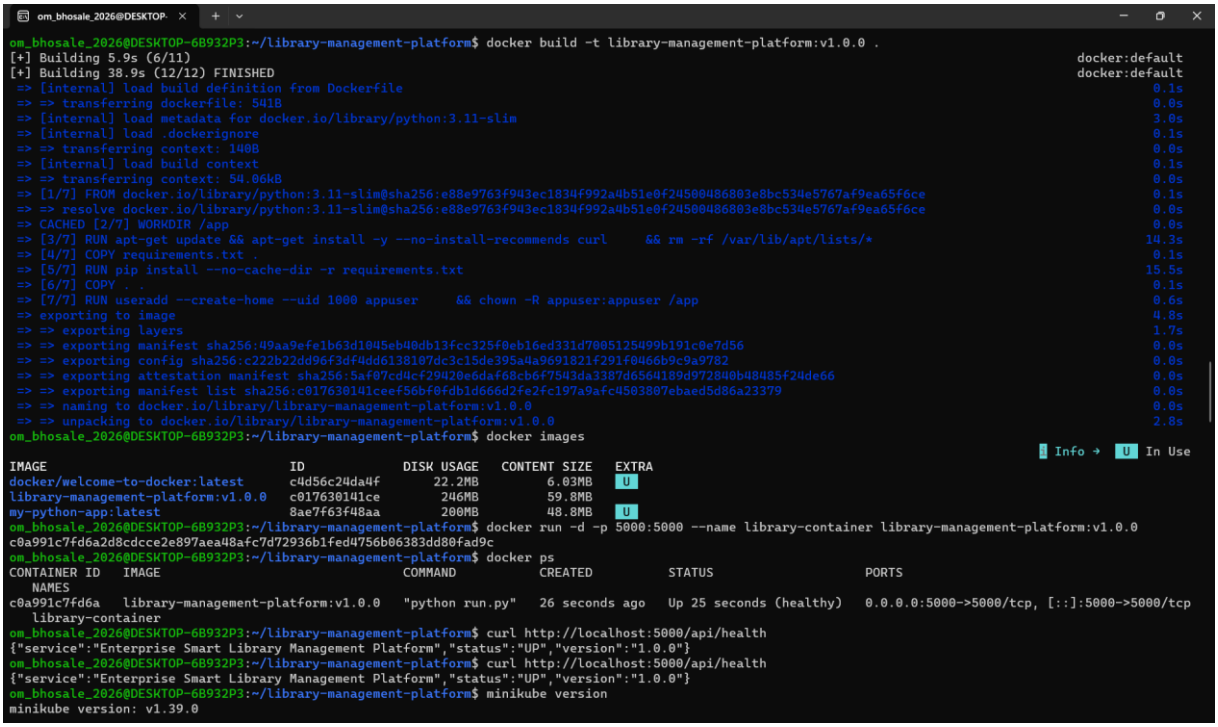


Figure 12: *Docker build, docker images, docker run, docker ps (Up – healthy) and curl health check; minikube version v1.39.0 is also shown*

Result: the application runs correctly inside a container, the built-in HEALTHCHECK marks it healthy, and the API is reachable from the host through the published port.

8. Topic 6 – Kubernetes Orchestration on Minikube

Kubernetes runs the container as a managed set of replicas and keeps the desired state automatically. A local cluster was created with Minikube using the Docker driver, and the locally built image was made available to the cluster.

8.1 Deployment procedure

```
minikube start --driver=docker
minikube image load library-management-platform:v1.0.0
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Resource	Configuration
Deployment library-deployment	3 replicas, label app=library-platform, RollingUpdate (maxSurge 1, maxUnavailable 0), image library-management-platform:v1.0.0 with imagePullPolicy IfNotPresent, liveness and readiness probes on /api/health:5000
Service library-service	Type NodePort, port 5000 → targetPort 5000, nodePort 30080

8.2 Verification of replicas and service

The deployment reached 3/3 ready, up-to-date and available replicas. All three pods were Running with 0 restarts. The service library-service was created as a NodePort with ClusterIP 10.102.114.144 and port mapping 5000:30080/TCP.

Minikube exposed the service at http://192.168.49.2:30080. Because the Docker driver is used on Linux, Minikube also opened a tunnel on a local port (127.0.0.1:42579 in Figure 13) that must stay open in its terminal while the service is used. The application was then tested from a second terminal through the generated URL, and /api/health returned status UP through Kubernetes.

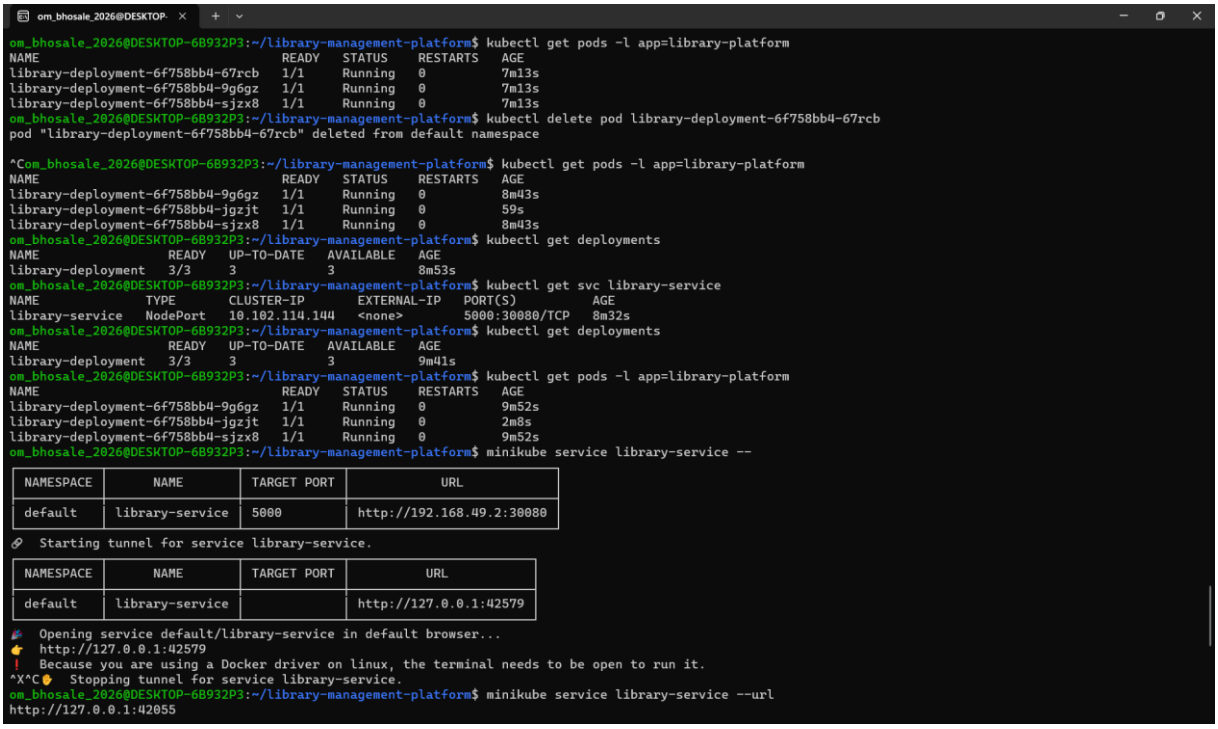


Figure 13: *kubectl get pods (3 Running), deletion of one pod, 3/3 deployment, NodePort service 5000:30080/TCP and minikube service URL*

8.3 Self-healing demonstration

To prove self-healing, a running pod was deleted and the pod list was checked again. Kubernetes immediately created a replacement pod so that the Deployment returned to the desired three replicas. This was demonstrated twice:

- **First test (Figure 13):** pod library-deployment-6f758bb4-67rcb was deleted; a new pod, library-deployment-6f758bb4-jgzjt, appeared (59 s old) alongside the two remaining pods, and the deployment stayed at 3/3.
- **Second test (Figure 14):** pod library-deployment-6f758bb4-9g6gz was deleted; the next listing shows it Terminating while a new pod, library-deployment-6f758bb4-4hc65 (16 s old), is already Running. Seconds later the cluster showed three Running pods again.

The final kubectl get all output lists three pods and one ReplicaSet (replicaset.apps/library-deployment-6f758bb4) with DESIRED 3, CURRENT 3 and READY 3, confirming that the ReplicaSet restored the replica count after each deletion.

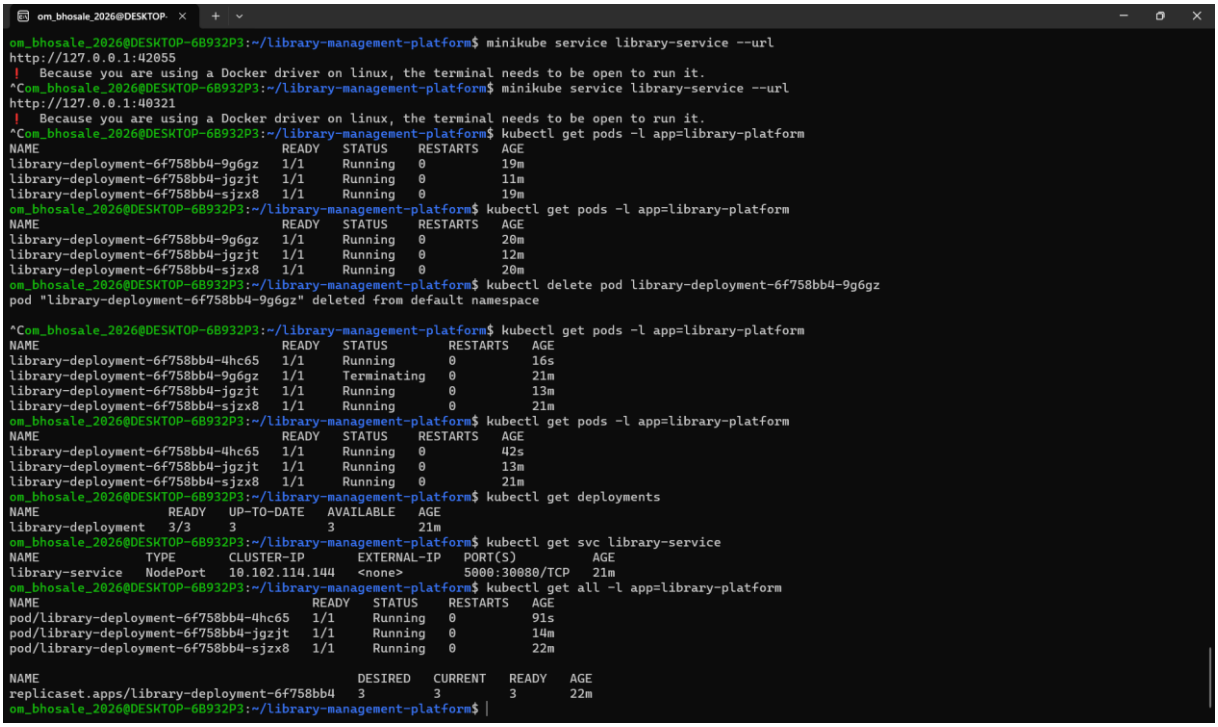


Figure 14: *Second self-healing test: deleted pod Terminating, replacement pod Running, deployment 3/3, service unchanged, ReplicaSet 3/3/3*

Result: the application ran as three healthy replicas behind a stable NodePort service, and Kubernetes automatically replaced deleted pods without any manual intervention.

9. Final Step – Shutdown and Preservation

After successful verification, the running services were stopped so that system resources were released while all project assets were kept.

Service	Command	Where to run
Minikube cluster	minikube stop (check with minikube status)	Ubuntu terminal
Docker container	docker stop library-container	Ubuntu terminal
Jenkins service	Stop-Service -Name Jenkins (check with Get-Service -Name Jenkins)	Windows PowerShell

Windows PowerShell commands such as Stop-Service must be run in PowerShell, not in Ubuntu, because Jenkins was installed on Windows. The following were preserved: the project directory ~/library-management-platform, the Docker image library-management-platform:v1.0.0, the Jenkins installation and job configuration, and the source code with the Kubernetes YAML files. Jenkins was stopped, not uninstalled.

10. Challenges Encountered and Resolutions

Challenge	What was observed	Outcome / handling
Jenkins apt repository key	apt-get update warned of NO_PUBKEY for pkg.jenkins.io (Figures 1–2)	Non-blocking for the practical; Ubuntu packages installed normally and Jenkins was run on Windows
Failed first Jenkins build	Build #1 failed (Figure 11)	Re-run after troubleshooting; build #2 passed all stages
Python version difference	Host runs Python 3.14.4 while the guide targets 3.11	Application worked on the host; the Docker image pins python:3.11-slim for consistency
Service access with Docker driver	Minikube warned that the terminal must stay open to keep the tunnel (Figure 13)	Tunnel left running in one terminal and the endpoint tested from another
Command environment	Windows service commands do not work in Ubuntu	Jenkins service control performed in PowerShell
Image availability in Minikube	Local image is not visible to the cluster by default	Image loaded with minikube image load, with imagePullPolicy IfNotPresent

11. Conclusion

The practical successfully demonstrated an end-to-end DevOps workflow. A modular Flask API was built and verified; an automated Pytest suite passed with 96% coverage; a Jenkins pipeline built the project and ran the tests successfully; the application was packaged as a non-root Docker image and ran as a healthy container; and Kubernetes on Minikube operated three replicas behind a NodePort service and automatically restored deleted pods. The exercise shows how automation, containers and orchestration improve reliability, repeatability and recovery in modern software delivery.

List of Figures

Figure	Description
1	Prerequisite installation and tool verification
2	Repeat verification in the project virtual environment
3	GitHub repository page
4	Source of app/routes/api.py
5	GET /api/health response
6	GET /api/books response
7	GET /api/users/U101 response
8	Pytest session start and dependency check
9	Pytest results and coverage report
10	Jenkins stage view, build #2
11	Jenkins job page and build history
12	Docker build, run, health check
13	Kubernetes pods, service and first self-healing test
14	Second self-healing test and ReplicaSet state